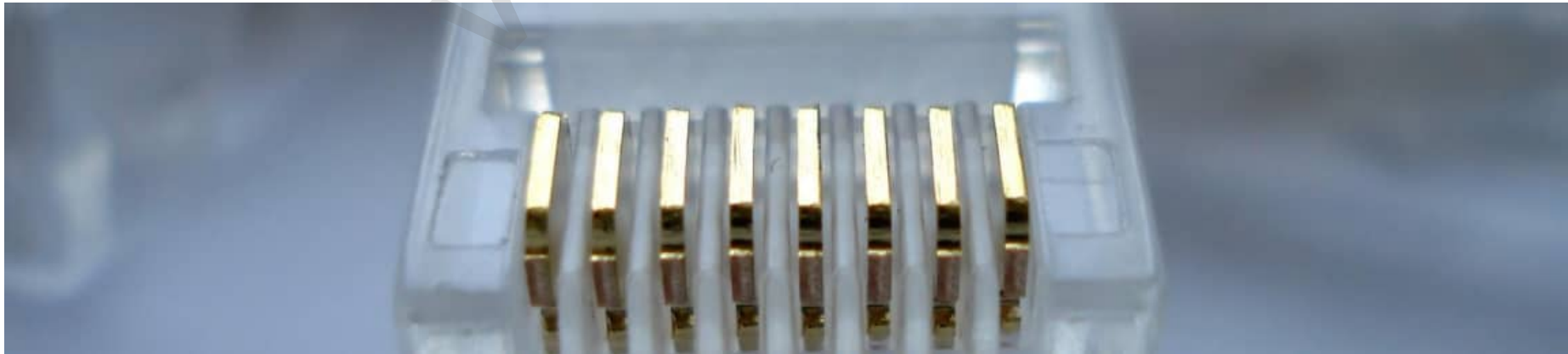


Module 1:

- i. Software and Software Engineering
- ii. Process Models



Overview

♦ **Software and Software Engineering:** The nature of Software, The unique nature of WebApps, Software Engineering, The software Process, The software Engineering practice, The software myths

Process Models: A generic process model, Process assessment and improvement, Prescriptive process models, Waterfall model, Incremental process models, Evolutionary process models, Concurrent models, Specialized process models, The Unified Process, Personal and Team Process Models.

By the end of this module, you will...

- ◆ Understand what software engineering is.
- ◆ Understand why software engineering is important.
- ◆ Know answers to key questions related to the software engineering discipline.

What is Software ?

Software is a program or set of programs containing instructions that provide desirable functions

What is Engineering ?

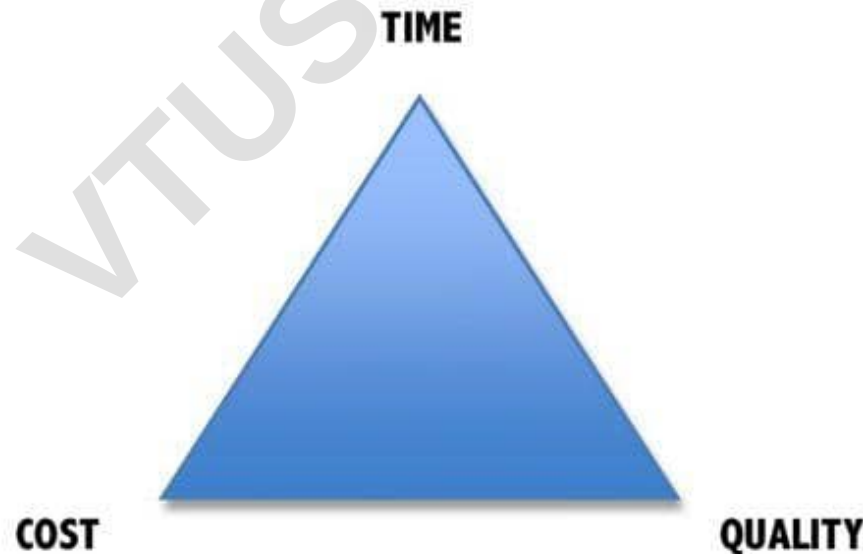
Application of science, tools and methods to find the cost effective solution to problems

What is Software Engineering?

Software Engineering is a systematic, disciplined and quantifiable approach used in the development, operation and maintenance of software

Why is Software Engineering important?

Complex systems need a disciplined approach for designing, developing and managing them.



Software Development Crises

Projects were:

- Late.
- Over budget.
- Unreliable.
- Difficult to maintain.
- Performed poorly.

Software errors....*the cost*

Errors in computer software can have devastating effects.

Software Crisis

Example 1: 2009, Computer glitch delays flights

Saturday 3rd October 2009-London, England (CNN)

- Dozens of flights from the UK were delayed Saturday after a glitch in an air traffic control system in Scotland, but the problem was fixed a few hours later.
- The **Air Traffic Service** agency said it reverted to backup equipment as engineering worked on the system.
- The problem did not create a safety issue but could cause delays in flights.
- Read more at:
<http://edition.cnn.com/2009/WORLD/europe/10/03/uk.flights.delayed>



Software Crisis

Example 2: Ariane 5 Explosion

- June 4th 1996, from French Guiana
- European Space Agency spent 10 years and \$7 billion to produce Ariane 5.
- Crash after 36.7 seconds, due to software bug in the rockets inertial Reference System.
- Caused by an overflow error. Trying to store a 64-bit number into a 16-bit space.
- Watch the video:
<http://www.youtube.com/watch?v=z-r9cYp3tTE>



Software Crisis

Example 3: 1992, London Ambulance Service

- Considered the largest ambulance service in the world.
- Overloaded problem.
- It was unable to keep track of the ambulances and their statuses. Sending multiple units to some locations and no units to other locations.
- Generates many exceptions messages.
- 46 deaths.



Therefore...

A well-disciplined approach to software development and management is necessary. This is called Software engineering.

Evolution of Software

- ♦ **1945-1960 II world war**
- ♦ **1960 – Software crisis**
- ♦ The term *software engineering* first appeared in the **1968** NATO Software Engineering Conference and was meant to provoke thought regarding what was then called the **“software crisis”**..
- ♦ **2000 – Internet**
- ♦ **2010 – AI, ML, DL**

1.1 The Nature of Software

Today software takes on a dual role

- **Software as a product**
 - Delivers computing potential across network of hardware
 - enables the Hardware to deliver the expected functionality.
 - acts as information transformer because it produces, manages, acquires, modifies, displays, or transmits information.
- **Software as a vehicle**
 - Used to deliver the product
 - Controls the computer & other programs (e.g., an operating system)
 - communication of information (e.g., networking software)
 - Helps build other software (e.g., software tools & environment)

1.1 The Nature of Software

- The role of computer software has undergone significant change over the last half-century.
- Dramatic improvements in hardware performance, computing architectures, memory and storage capacity, exotic input and output options, have all precipitated more sophisticated and complex computer-based systems.
- Problems for those who must build complex systems.
- Today, a huge software industry has become a dominant factor in the economies of the industrialized world.
- Teams of software specialists, each focusing on one part of the technology required to deliver a complex application, have replaced the lone programmer of an earlier era.

1.1 The Nature of Software

- yet, the questions that were asked of the lone programmer are the same questions that are asked when modern computer-based systems are built
 - I. Why does it take so long to get software finished?
 - II. Why are development costs so high?
 - III. Why can't we find all errors before we give the software to our customers?
 - IV. Why do we spend so much time and effort maintaining existing programs?
 - V. Why do we continue to have difficulty in measuring progress as software is being developed and maintained?
- These, and many other questions, are a manifestation of the concern about software and the manner in which it is developed—a concern that has lead to the adoption of software engineering practice.

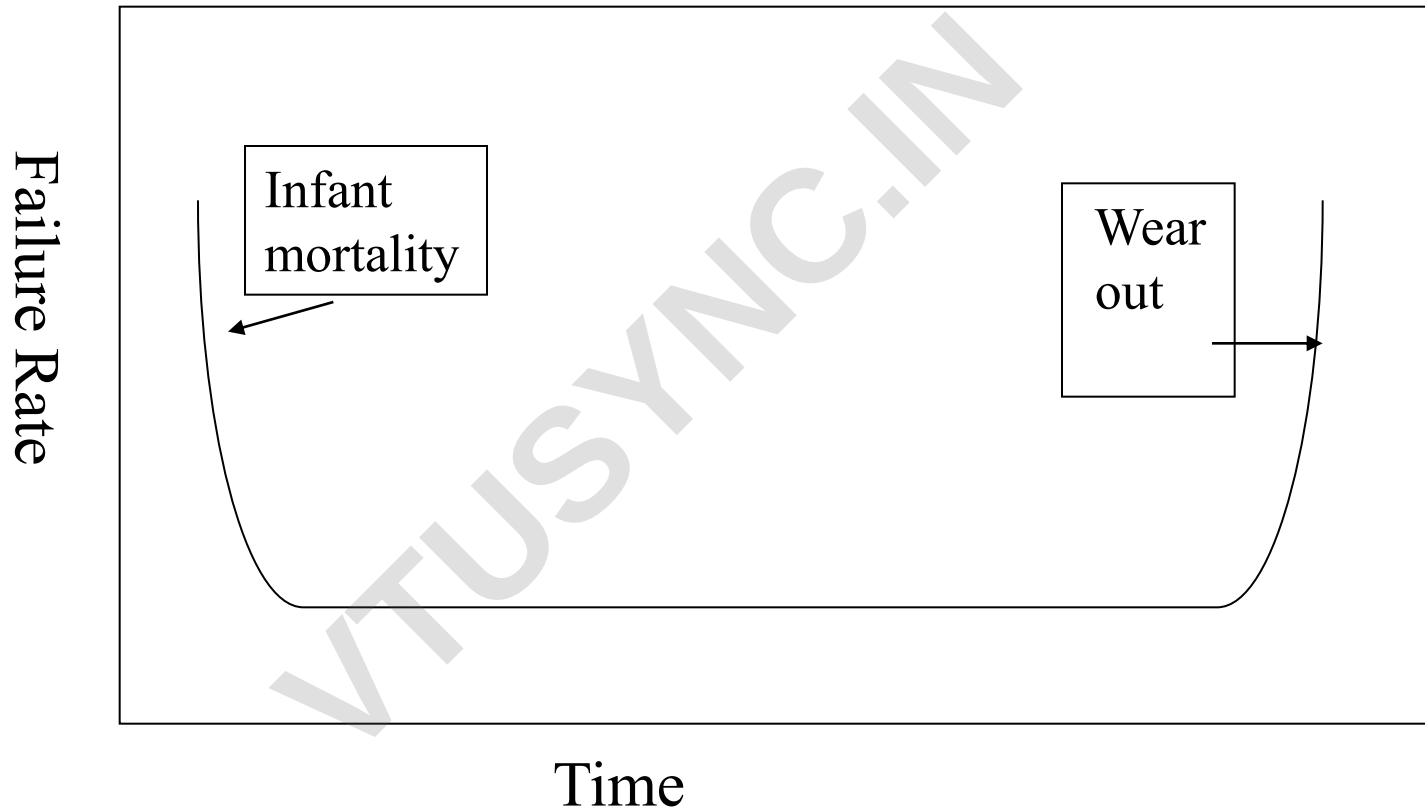
1.1.1 Defining Software?

- 1) instructions (programs) that when executed provide desired function and performance
 - 2) data structures that enable the programs to adequately manipulate information
 - 3) documents that describe the operation and use of the programs
- For better understanding, it's important to examine the characteristics of software that make it different from other things that human beings build.
 - Software is a logical rather than physical system element

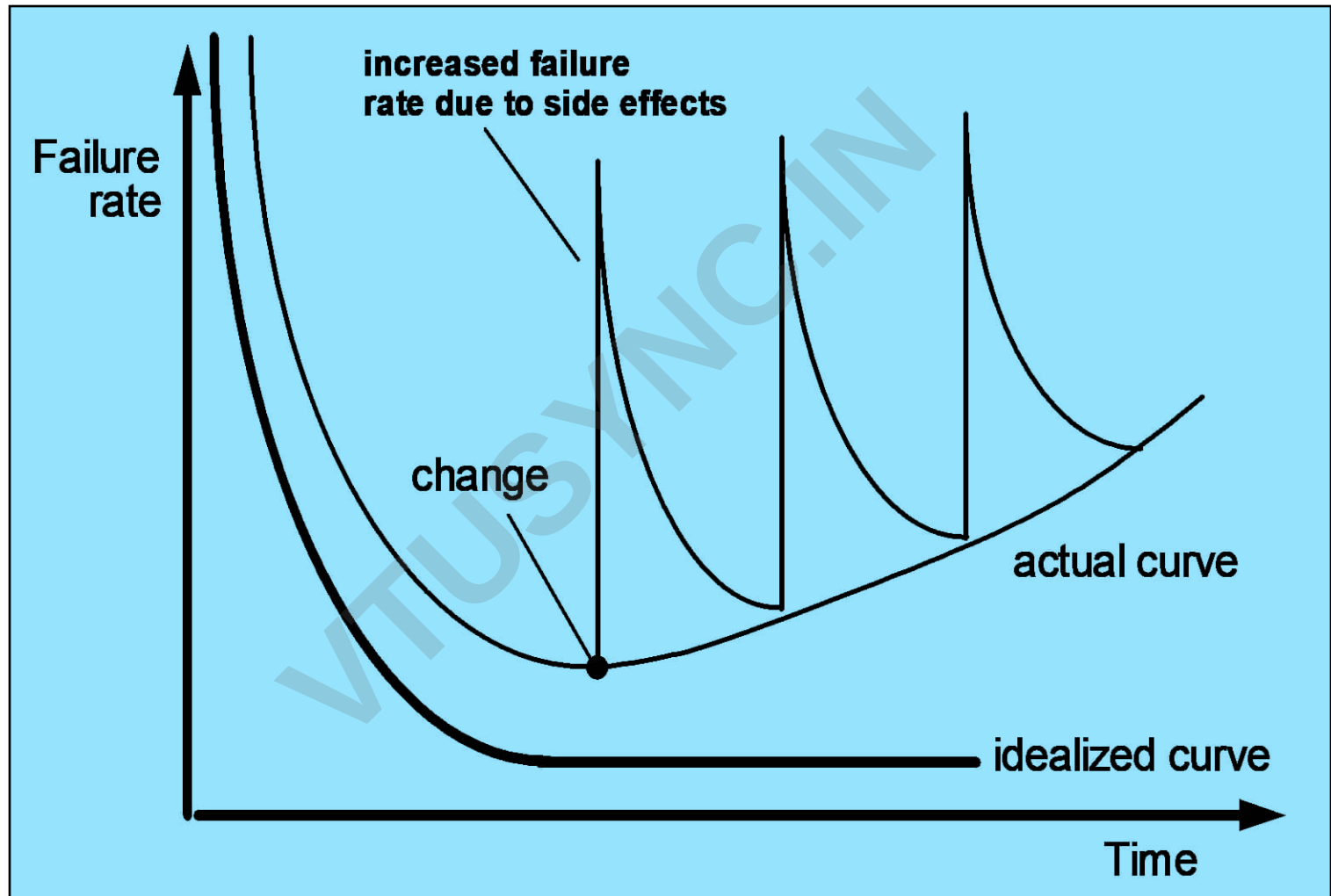
Characteristics of Software

- 1. Software is developed or engineered; it is not manufactured in the classical sense.*
- 2. Software doesn't "wear out."*
- 3. Although the industry is moving toward component-based construction, most **software** continues to be **custom built**.*

Failure (“Bathtub”) Curve for Hardware



Wear vs. Deterioration



Types of Software

- **Generic products.**
 - Stand-alone systems that are marketed and sold to any customer who wishes to buy them.
 - Examples – PC software such as graphics programs, project management tools; CAD software; software for specific markets such as appointments systems for dentists.
 - The specification of what the software should do is owned by the software developer and decisions on software change are made by the developer.
- **Customized or bespoke products.**
 - Software that is commissioned by a specific customer to meet their own needs.
 - Examples – embedded control systems, air traffic control software, traffic monitoring systems.
 - The specification of what the software should do is owned by the customer for the software and they make decisions on software changes that are required.

1.1.2 Changing nature of software/ Software Application Domains

1. System software:

- a collection of programs written to service other programs.
- system software (e.g., compilers, editors, and file management utilities) processes complex, but determinate information structures.
- systems applications (e.g., operating system components, drivers, networking software, telecommunications processors) process largely indeterminate data.
- In either case, the systems software area is characterized by **heavy interaction with computer hardware**; **heavy usage by multiple users**; concurrent operation that requires **scheduling, resource sharing, and sophisticated process management**; **complex data structures**; and **multiple external interfaces**.

1.1.2 Changing nature of software/ Software Application Domains

2. Application software

- Program or group of programs designed for end users. Eg: MS word, PP, Excel, Email app etc

3. Engineering/scientific software—

- has been characterized by “number crunching” algorithms.
- Applications range from astronomy to volcanology, from automotive stress analysis to space shuttle orbital dynamics, and from molecular biology to automated manufacturing.CAD

1.1.2 Changing nature of software/ Software Application Domains

3. Embedded software

- **resides within a product or system and is used to** implement and control features and functions for the end user and for the system itself
- Embedded software can perform limited and esoteric functions (e.g., key pad control for a microwave oven) or provide significant function and control capability (e.g., digital functions in an automobile such as fuel control, dashboard displays, and braking systems).

4. Product-line software

- **designed to provide a specific capability for use by** many different customers.
- Product-line software can focus on a limited and esoteric marketplace (e.g., inventory control products) or address mass consumer markets (e.g., word processing, spreadsheets, computer graphics, multimedia, entertainment, database management, and personal and business financial applications).

1.1.2 Changing nature of software/ Software Application Domains

5. Web applications—called “WebApps,”

- **this network-centric software category** spans a wide array of applications.
- In their simplest form, WebApps can be little more than a set of linked hypertext files that present information using text and limited graphics.
- However, as Web 2.0 emerges, WebApps are evolving into sophisticated computing environments that not only provide stand-alone features, computing functions, and content to the end user, but also are integrated with corporate databases and business applications.

6. Artificial intelligence software

- **makes use of non numerical algorithms to** solve complex problems that are not amenable to computation or straightforward analysis.
- Applications within this area include robotics, expert systems, pattern recognition (image and voice), artificial neural networks, theorem proving, and game playing.

1.1.2 Changing nature of software/ Software Application Domains

- new challenges that have appeared on the horizon:
 1. **Open-world computing—the rapid growth of wireless networking** may soon lead to true pervasive, distributed computing. The challenge for software engineers will be to develop systems and application software that will allow mobile devices, personal computers, and enterprise systems to communicate across vast networks.
 2. **Netsourcing—the World Wide Web is rapidly becoming a computing engine** as well as a content provider. The challenge for software engineers is to architect simple (e.g., personal financial planning) and sophisticated applications that provide a benefit to targeted end-user markets worldwide.
 3. **Open source—a growing trend that results in distribution of source code** for systems applications so that many people can contribute to its development. The challenge for software engineers is to build source code that is self-descriptive, but more importantly, to develop techniques that will enable both customers and developers to know what changes have been made and how those changes manifest themselves within the software.

1.1.3 Legacy Software

- Hundreds of thousands of computer programs fall into one of the seven broad application domains.
- Some of these are state of- the-art software—just released to individuals, industry, and government. But other programs are older, in some cases *much older*.
- These older programs—often referred to as *legacy software*—*have been the focus of continuous attention* and concern since the 1960s.
- The proliferation of such systems is causing headaches for large organizations who find them costly to maintain and risky to evolve.
- *Legacy system have poor quality, inextensible designs, convoluted code, poor or nonexistent documentation, test cases and results that were never archived, a poorly managed change history—the list can be quite long.*
- And yet, these systems support “core business functions and are indispensable to the business.” What to do?
- DO NOTHING, However, as time passes, legacy systems often evolve for one or more of the following reasons:

Legacy Software

Why must it change?

- software must be **adapted** to meet the needs of new computing environments or technology.
- software must be **enhanced** to implement new business requirements.
- software must be **extended to make it interoperable** with other more modern systems or databases.
- software must be **re-architected** to make it viable within a network environment.

1.2 The Unique Nature of WEBAPPS

- In the early days of the World Wide Web (circa 1990 to 1995), *websites consisted of* little more than a set of linked hypertext files that presented information using text and limited graphics.
- As time passed, the augmentation of HTML by development tools (e.g., XML, Java) enabled Web engineers to provide computing capability along with informational content.
- ***Web-based systems and applications*** (referred to these collectively as *WebApps*) were born.
- *Today, WebApps have evolved into sophisticated* computing tools that not only provide stand-alone function to the end user, but also have been integrated with corporate databases and business applications.

Characteristics of WebApps - I

- **Network intensiveness.** A WebApp resides on a network and must serve the needs of a diverse community of clients.
- **Concurrency.** A large number of users may access the WebApp at one time.
- **Unpredictable load.** The number of users of the WebApp may vary by orders of magnitude from day to day.
- **Performance.** If a WebApp user must wait too long (for access, for server-side processing, for client-side formatting and display), he or she may decide to go elsewhere.
- **Availability.** Although expectation of 100 percent availability is unreasonable, users of popular WebApps often demand access on a "24/7/365" basis.

Characteristics of WebApps - II

- **Data driven.** The primary function of many WebApps is to use hypermedia to present text, graphics, audio, and video content to the end-user. Eg: e-commerce or financial application
- **Content sensitive.** The quality and aesthetic nature of content remains an important determinant of the quality of a WebApp.
- **Continuous evolution.** Unlike conventional application software that evolves over a series of planned, chronologically-spaced releases, Web applications evolve continuously.
- **Immediacy.** Although *immediacy*—the compelling need to get software to market quickly—is a characteristic of many application domains, WebApps often exhibit a time to market that can be a matter of a few days or weeks.
- **Security.** Because WebApps are available via network access, it is difficult, if not impossible, to limit the population of end-users who may access the application.
- **Aesthetics.** An undeniable part of the appeal of a WebApp is its look and feel.

1.3 Software Engineering

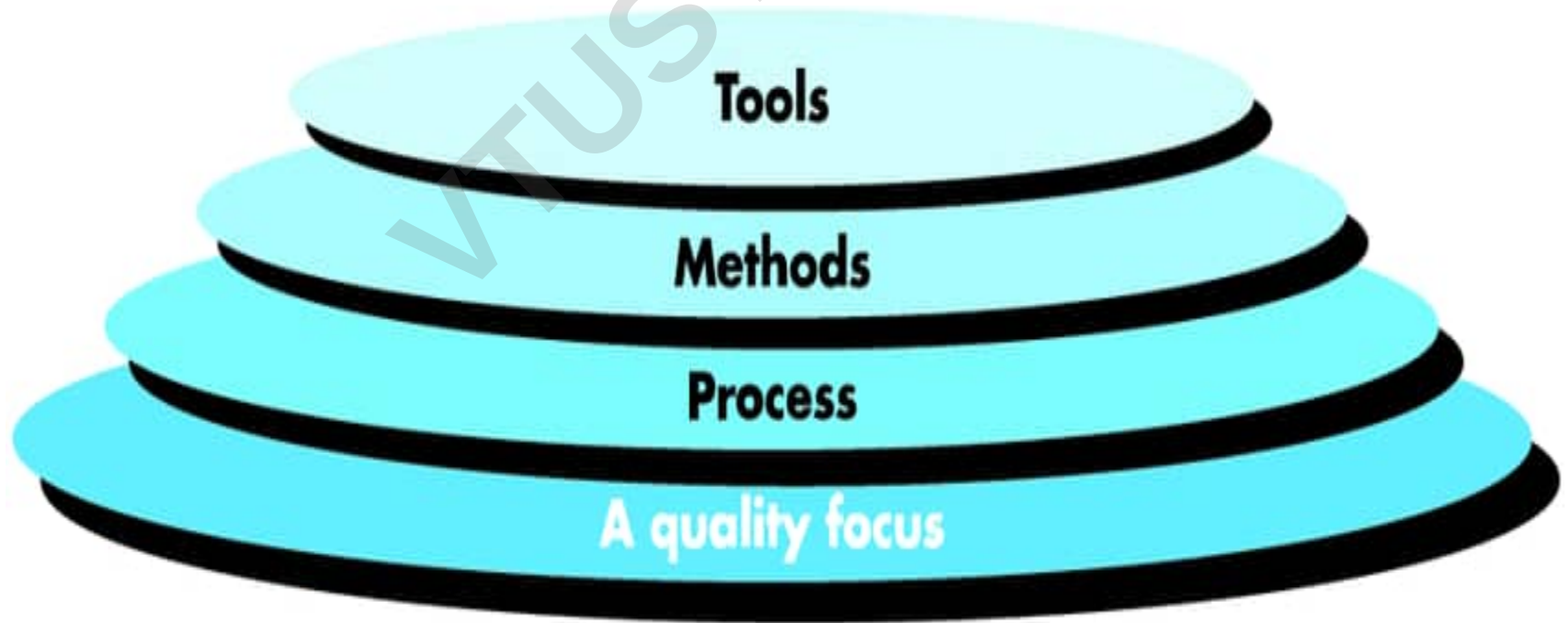
- In order to build software that is ready to meet the challenges of the twenty-first century, you must recognize a few simple realities:
 1. ***a concerted effort should be made to understand the problem before a software solution is developed.***
 2. ***Design is a pivotal software engineering activity.***
 3. ***software should exhibit high quality***
 4. ***software should be maintainable.***
- These simple realities lead to one conclusion: *software in all of its forms and across all of its application domains should be engineered And that leads us to the topic of this subject—software engineering.*
- a definition proposed by Fritz Bauer [Nau69] at the seminal conference on the subject still serves as a basis for discussion: **[Software engineering is] the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.**

Software Engineering

- The IEEE definition:
 - *Software Engineering: (1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software. (2) The study of approaches as in (1).*

Software Engineering(Conti..)

- And yet, a “systematic, disciplined, and quantifiable” approach applied by one software team may be burdensome to another.
- We need discipline, but we also need adaptability and agility.
- Software engineering is a layered technology



Software Engineering(Conti..)

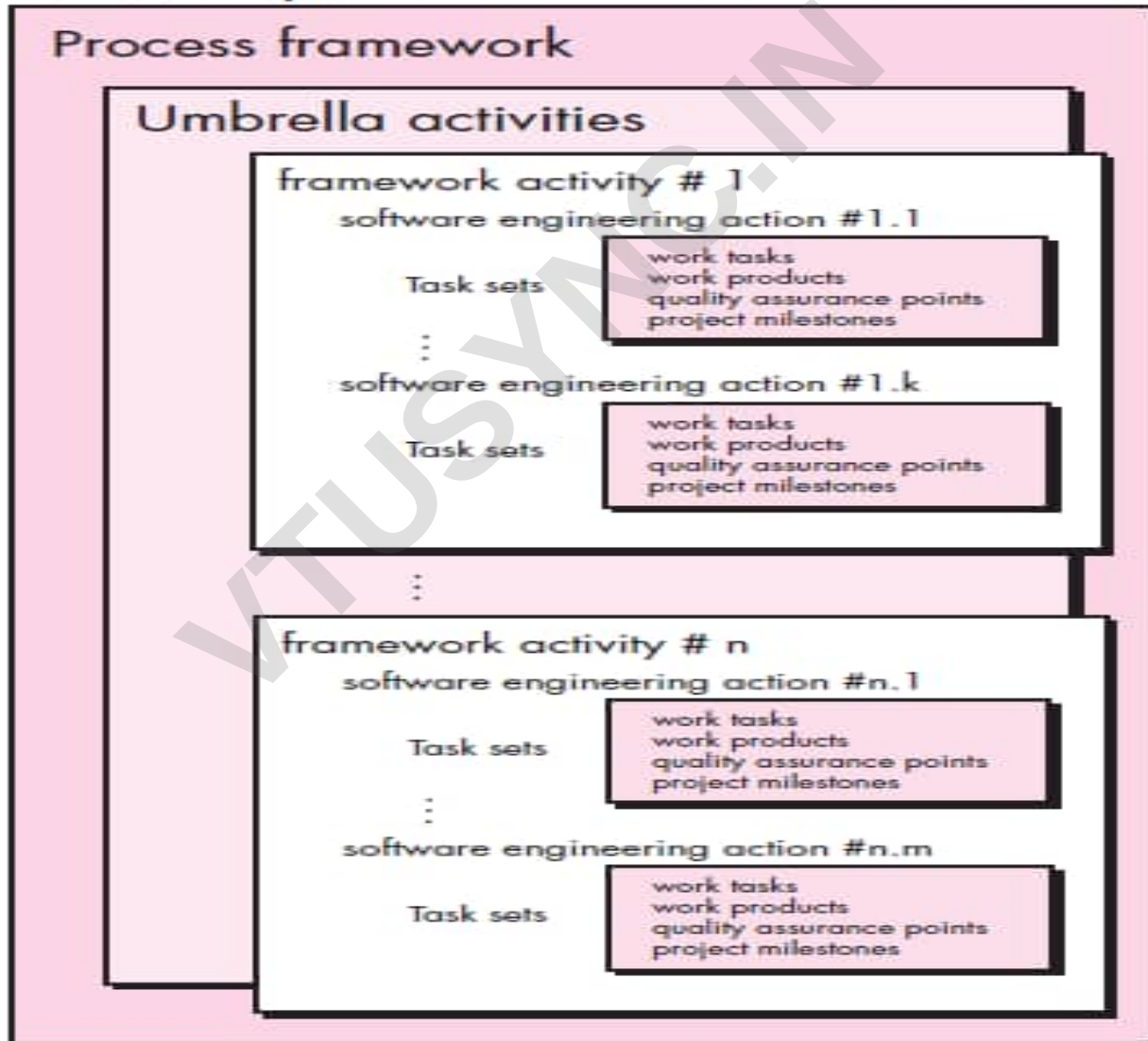
- Any engineering approach (including software engineering) must rest on an **organizational commitment to quality**. The bedrock that supports software engineering is a **quality focus**.
- **The foundation for software engineering is the *process layer***. Process defines a framework that must be established for effective delivery of software engineering technology.
- **The software process forms the basis for management control of software projects** and establishes the context in which technical methods are applied, work products (models, documents, data, reports, forms, etc.) are produced, milestones are established, quality is ensured, and change is properly managed.
- **Software engineering *methods* provide the technical how-to's for building software.**

Methods encompass a broad array of tasks that include **communication, requirements analysis, design modeling, program construction, testing, and support**.

- Software engineering ***tools* provide automated or semi automated support for the process and the methods**. Tools are integrated ,
- A system for the support of software development, called ***computer-aided software engineering, is established***.

1.4 The Software Process

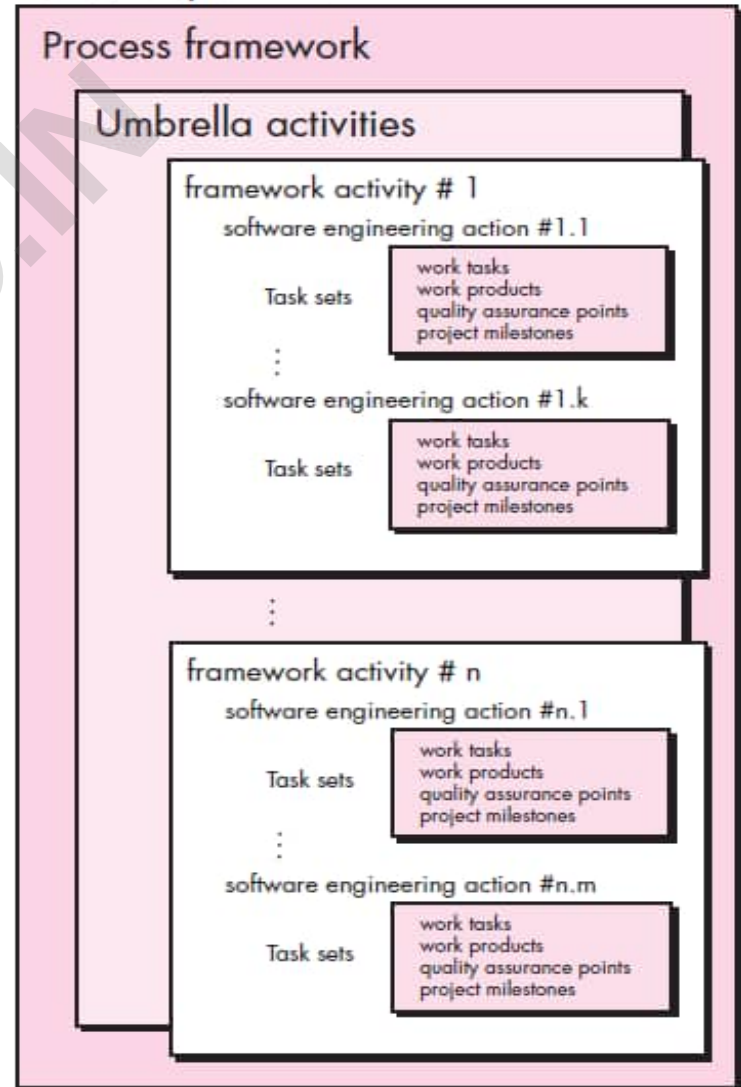
Software process



1.4 The Software Process

- A *process* is a collection of **activities, actions, and tasks** that are performed when some work product is to be created.
- An **activity** strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied.
- An **action** (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., an architectural design model).
- A **task** focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome.

Software process



1.4 The Software Process

- In the context of software engineering, a process is *not a rigid prescription for how* to build computer software.
- Rather, it is an adaptable approach that enables the people doing the work (the software team) to pick and choose the appropriate set of work actions and tasks.
- A process framework establishes the foundation for a complete software engineering process by identifying a small number of *framework activities that are applicable* to all software projects, regardless of their size or complexity.
- In addition, the process framework encompasses a set of *umbrella activities that are applicable* across the entire software process

1.4 The Software Process

A generic process framework for software engineering encompasses five activities:

- **Communication:** The intent is to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.
- **Planning:** the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule
- **Modeling:** models helps to better understand software requirements and the design that will achieve those requirements.
- **Construction:** This activity combines code generation (either manual or automated) and the testing that is required to uncover errors in the code.
- **Deployment:** The software (as a complete entity or as a partially completed increment) is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation.

1.4 The Software Process

umbrella activities: *In general, umbrella activities are applied throughout a software*

project and help a software team **manage and control** progress, quality, change, and risk. Typical umbrella activities include:

- **Software project tracking and control**—allows the software team to assess progress against the project plan and take any necessary action to maintain the schedule.
- **Risk management**—assesses risks that may affect the outcome of the project or the quality of the product.
- **Software quality assurance**—defines and conducts the activities required to ensure software quality.
- **Technical reviews**—assesses software engineering work products in an effort to uncover and remove errors before they are propagated to the next activity.

- **Measurement**—defines and collects process, project, and product measures that assist the team in delivering software that meets stakeholders' needs; can be used in conjunction with all other framework and umbrella activities.
- **Software configuration management**—manages the effects of change throughout the software process.
- **Reusability management**—defines criteria for work product reuse (including software components) and establishes mechanisms to achieve reusable components.
- **Work product preparation and production**—encompasses the activities required to create work products such as models, documents, logs, forms, and lists.

a process adopted for one project might be significantly different than a process adopted for another project. Among the differences are

Adapting a Process Model

- the overall flow of activities, actions, and tasks and the interdependencies among them
- the degree to which actions and tasks are defined within each framework activity
- the degree to which work products are identified and required
- the manner which quality assurance activities are applied
- the manner in which project tracking and control activities are applied
- the overall degree of detail and rigor with which the process is described
- the degree to which the customer and other stakeholders are involved with the project
- the level of autonomy given to the software team
- the degree to which team organization and roles are prescribed

1.5 The Software Engineering Practice

- In Previous Section, I introduced a generic software process model composed of a set of activities that establish a framework for software engineering practice.
- Generic framework activities—**communication, planning, modeling, construction, and deployment**—and **umbrella activities** establish a skeleton architecture for software engineering work.
- But how does the practice of software engineering fit in?
- **In the sections that follow, you'll gain a basic understanding of the generic concepts and principles that apply to framework activities.**

The Essence of Practice

Polya outlined the essence of problem solving, and consequently, the essence of software engineering practice:

■ Polya suggests:

1. *Understand the problem* (communication and analysis).
2. *Plan a solution* (modeling and software design).
3. *Carry out the plan* (code generation).
4. *Examine the result for accuracy* (testing and quality assurance).

In the context of software engineering, these commonsense steps lead to a series of essential questions

Understand the Problem

- *Who has a stake in the solution to the problem?* That is, who are the stakeholders?
- *What are the unknowns?* What data, functions, and features are required to properly solve the problem?
- *Can the problem be compartmentalized?* Is it possible to represent smaller problems that may be easier to understand?
- *Can the problem be represented graphically?* Can an analysis model be created?

Plan the Solution

slow down just a bit and do a little design:

- *Have you seen similar problems before?* Are there patterns that are recognizable in a potential solution? Is there existing software that implements the data, functions, and features that are required?
- *Has a similar problem been solved?* If so, are elements of the solution reusable?
- *Can subproblems be defined?* If so, are solutions readily apparent for the subproblems?
- *Can you represent a solution in a manner that leads to effective implementation?* Can a design model be created?

Carry Out the Plan

the “plan” will allow you to proceed without getting lost.

- *Does the solution conform to the plan?* Is source code traceable to the design model?
- *Is each component part of the solution provably correct?* Has the design and code been reviewed, or better, have correctness proofs been applied to algorithm?

You can't be sure that your solution is perfect, but you can be sure that you've designed a sufficient number of tests to uncover as many errors as possible.

Examine the Result

- *Is it possible to test each component part of the solution?* Has a reasonable testing strategy been implemented?
- *Does the solution produce results that conform to the data, functions, and features that are required?* Has the software been validated against all stakeholder requirements?

1.5.2 General Principles

- The dictionary defines the word *principle* as “an important underlying law or assumption required in a system of thought.”
- David Hooker [Hoo96] has proposed seven principles that focus on software engineering practice as a whole. They are

1. **The First Principle: *The Reason It All Exists***

- A software system exists for one reason: *to provide value to its users.*

2. **The Second Principle: *KISS (Keep It Simple, Stupid!)***

- *All design should be as simple as possible, but no simpler*

3. **The Third Principle: *Maintain the Vision***

4. **The Fourth Principle: *What You Produce, Others Will Consume***

5. **The Fifth Principle: *Be Open to the Future***

A system with a long lifetime has more value.

6. **The Sixth Principle: *Plan Ahead for Reuse***

- *Planning ahead for reuse reduces the cost and increases the value of both the reusable components and the systems into which they are incorporated.*

7. **The Seventh principle: *Think!***

- *Placing clear, complete thought before action almost always produces better results.*

1.6 The Software MYTHS

- Software myths—erroneous beliefs about software and the process that is used to build it
- can be traced to the earliest days of computing.
- Myths have a number of attributes that make them insidious. For instance, they appear to be reasonable statements of fact (sometimes containing elements of truth), they have an intuitive feel,
- and they are often promoted by experienced practitioners who “know the score.”
- However, old attitudes and habits are difficult to modify, and remnants of software myths remain.

1.6 The Software MYTHS

1. Management myths.

Myth: *We already have a book that's full of standards and procedures for building software. Won't that provide my people with everything they need to know?*

Reality: The book of standards may very well exist, but is it used? Are software practitioners aware of its existence? Does it reflect modern software engineering practice? Is it complete? Is it adaptable? Is it streamlined to improve time-to-delivery while still maintaining a focus on quality? In many cases, the answer to all of these questions is "no."

Myth: *If we get behind schedule, we can add more programmers and catch up (sometimes called the "Mongolian horde" concept).*

Reality: Software development is not a mechanistic process like manufacturing. In the words of Brooks [Bro95]: "adding people to a late software project makes it later." At first, this statement may seem counterintuitive. However, as new people are added, people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort. People can be added but only in a planned and well-coordinated manner.

Myth: *If I decide to outsource the software project to a third party, I can just relax and let that firm build it.*

Reality: If an organization does not understand how to manage and control software projects internally, it will invariably struggle when it outsources software projects.

2. Customer myths

Myth: *A general statement of objectives is sufficient to begin writing programs—we can fill in the details later.*

Reality: Although a comprehensive and stable statement of requirements is not always possible, an ambiguous “statement of objectives” is a recipe for disaster. Unambiguous requirements (usually derived

iteratively) are developed only through effective and continuous communication between customer and developer.

Myth: *Software requirements continually change, but change can be easily accommodated because software is flexible.*

Reality: It is true that software requirements change, but the impact of change varies with the time at which it is introduced. When requirements changes are requested early (before design or code has been started), the cost impact is relatively small.¹⁶ However, as time passes, the cost impact grows rapidly—resources have been committed, a design framework has been established, and change can cause upheaval that requires additional resources and major design modification.

3. Practitioner's myths.

Myth: *Once we write the program and get it to work, our job is done.*

Reality: Someone once said that “the sooner you begin ‘writing code,’ the longer it’ll take you to get done.” Industry data indicate that between 60 and 80 percent of all effort expended on software will be expended after it is delivered to the customer for the first time.

Myth: *Until I get the program “running” I have no way of assessing its quality.*

Reality: One of the most effective software quality assurance mechanisms can be applied from the inception of a project—the *technical review*. Software reviews (described in Chapter 15) are a “quality filter” that have been found to be more effective than testing for finding certain classes of software defects.

Myth: *The only deliverable work product for a successful project is the working program.*

Reality: A working program is only one part of a software configuration that includes many elements. A variety of work products (e.g., models, documents, plans) provide a foundation for successful engineering and, more important, guidance for software support.

Myth: *Software engineering will make us create voluminous and unnecessary documentation and will invariably slow us down.*

Reality: Software engineering is not about creating documents. It is about creating a quality product. Better quality leads to reduced rework. And reduced rework results in faster delivery times.



How a Project Starts

The scene: Meeting room at CPI

Corporation, a (fictional) company that makes consumer products for home and commercial use.

The players: Mal Golden, senior manager, product development; Lisa Perez, marketing manager; Lee Warren, engineering manager; Joe Camalleri, executive VP, business development

The conversation:

Joe: Okay, Lee, what's this I hear about your folks developing a what? A generic universal wireless box?

Lee: It's pretty cool . . . about the size of a small matchbook . . . we can attach it to sensors of all kinds, a digital camera, just about anything. Using the 802.11g wireless protocol. It allows us to access the device's output without wires. We think it'll lead to a whole new generation of products.

Joe: You agree, Mal?

Mal: I do. In fact, with sales as flat as they've been this year, we need something new. Lisa and I have been doing a little market research, and we think we've got a line of products that could be big.

Joe: How big . . . bottom line big?

Mal (avoiding a direct commitment): Tell him about our idea, Lisa.

Lisa: It's a whole new generation of what we call "home management products." We call 'em *SafeHome*. They use the new wireless interface, provide homeowners or small-business people with a system that's controlled by their PC—home security, home surveillance, appliance and device control—you know, turn down the home air conditioner while you're driving home, that sort of thing.

Lee (jumping in): Engineering's done a technical feasibility study of this idea, Joe. It's doable at low manufacturing cost. Most hardware is off-the-shelf. Software is an issue, but it's nothing that we can't do.

Joe: Interesting. Now, I asked about the bottom line.

Mal: PCs have penetrated over 70 percent of all households in the USA. If we could price this thing right, it could be a killer-App. Nobody else has our wireless box . . . it's proprietary. We'll have a 2-year jump on the competition. Revenue? Maybe as much as 30 to 40 million dollars in the second year.

Joe (smiling): Let's take this to the next level. I'm interested.

Process Models

2.1 A Generic Process Model

Software Process includes:

Tasks: They focus on a small, specific objective.

Action: It is a set of tasks that produce a major work product.

Activities: Activities are groups of related tasks and actions for a major objective.

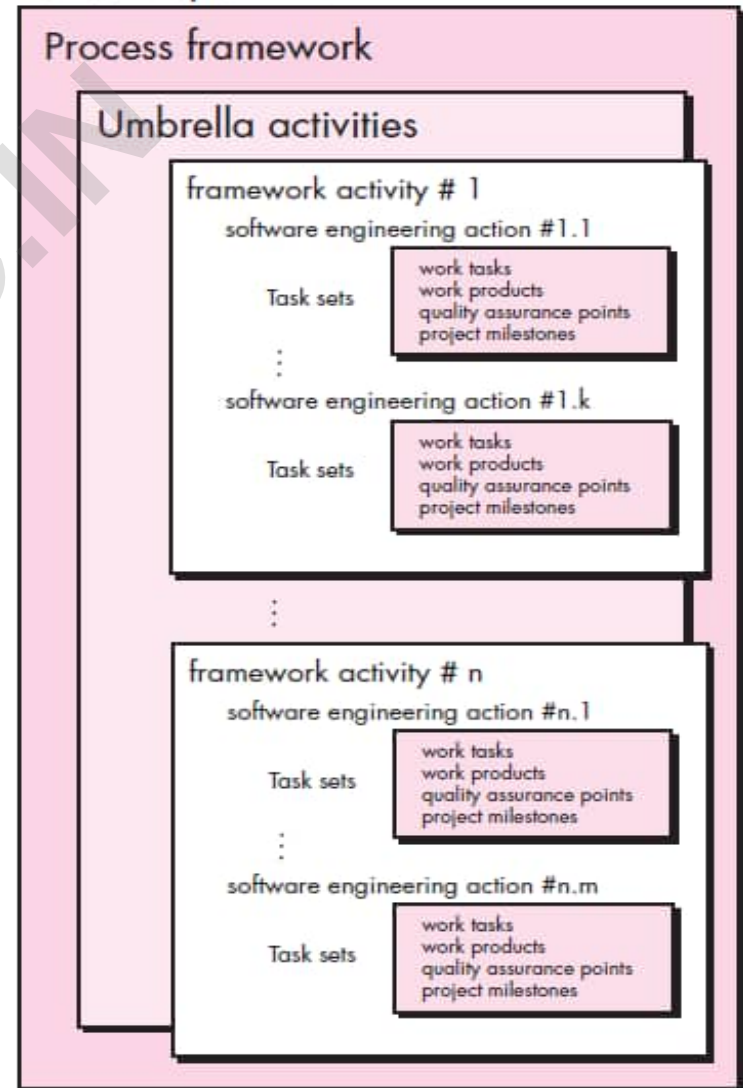
Process Framework Activities

1. Communication
2. Planning
3. Modeling- Analysis of requirement, design
4. Construction- Coding, testing
5. Deployment

Umbrella Activities

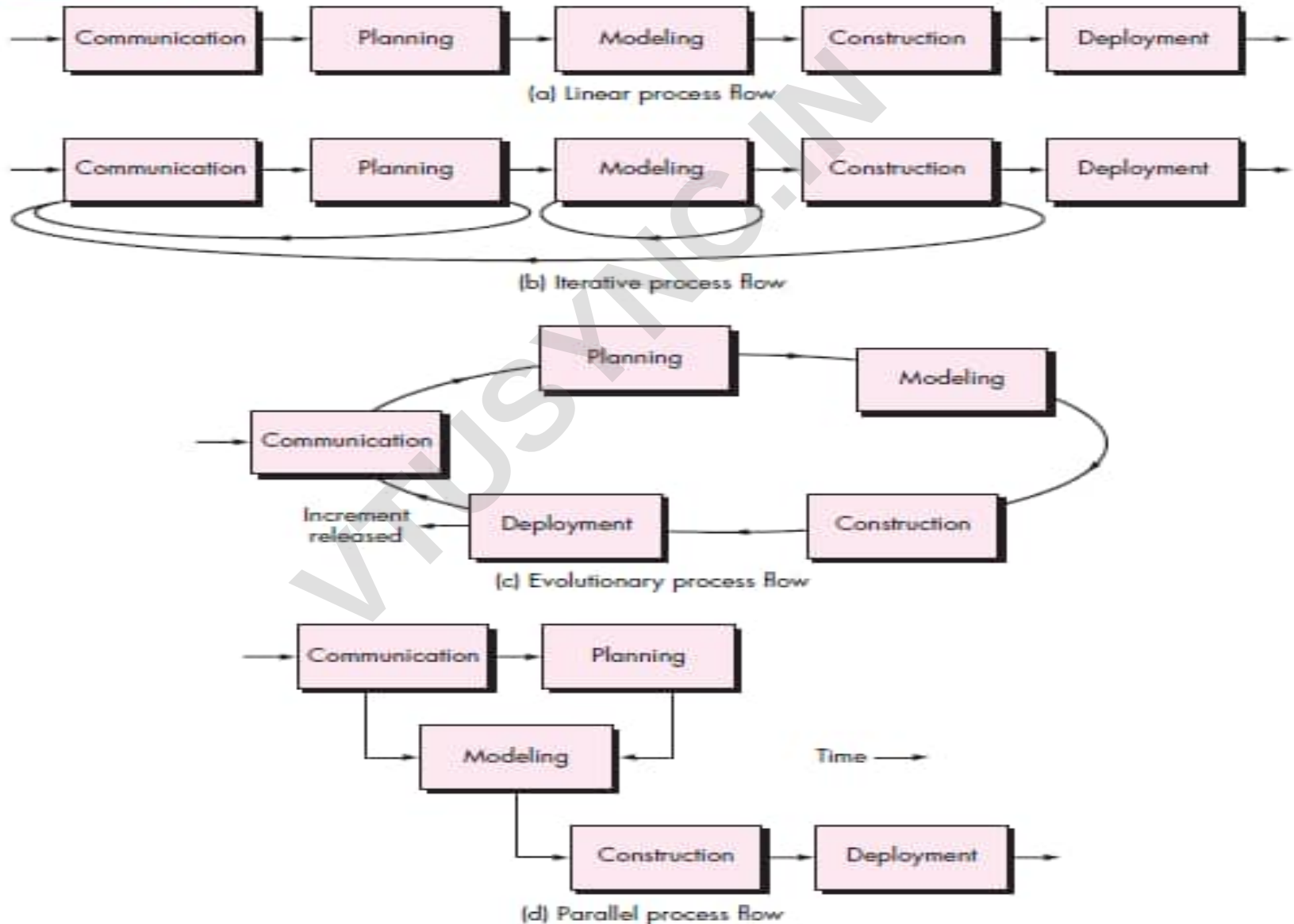
1. Project tracking and control
2. Risk management
3. Quality assurance
4. Configuration management
5. Technical Review etc

Software process



Process Flow

FIGURE 2.2 Process flow



2.1.1 Defining a Framework Activity

- A software team would need significantly more information before it could properly execute any one of these activities as part of the software process.
- Therefore, you are faced with a key question: **1) *What actions appropriate for a framework activity* 2) *given the nature of the problem to be solved, the characteristics of the people doing the work* 3) *and the stakeholders who are sponsoring the project?***
- For a small software project requested by one person (at a remote location) with simple, straightforward requirements, the communication activity might encompass little more than a phone call with the appropriate stakeholder.
- Therefore, the only necessary action is *phone conversation, and the work tasks (the task set) that this action encompasses* are:
 - 1. Make contact with stakeholder via telephone.**
 - 2. Discuss requirements and take notes.**
 - 3. Organize notes into a brief written statement of requirements.**
 - 4. E-mail to stakeholder for review and approval.**
- If the project was considerably more complex with many stakeholders, each with a different set of (sometime conflicting) requirements, the communication activity might have six distinct actions : ***inception, elicitation, elaboration, negotiation, specification, and validation.***

2.1.2 Identifying a Task Set

- Each software engineering action (e.g., *elicitation*, an action associated with the communication activity) can be represented by a number of different *task sets*—
- *each task set a collection of software engineering work tasks, related work products, quality assurance points, and project milestones.*
- You should choose a task set that best accommodates the needs of the project and the characteristics of your team.
- This implies that a software engineering action can be adapted to the specific needs of the software project and the characteristics of the project team.

2.1.3 Process Patterns

- A *process pattern* describes a process-related problem that is encountered during software engineering work, identifies the environment in which the problem has been encountered, and suggests one or more proven solutions to the problem
- Patterns can be defined at any level of abstraction. Used to describe problem (solution) associated with Process model(Eg: Prototyping) or framework activity(Eg: planning) or an action within framework activity (Eg: project estimation)
- Ambler [Amb98] has proposed a template for describing a process pattern:
 1. **Pattern Name:** The pattern is given a meaningful name describing it within the context of the software process (e.g., Technical Reviews).
 2. **Forces:** The environment in which the pattern is encountered and the issues that make the problem visible and may affect its solution.

Process Patterns (Conti)

3. Type: The pattern type is specified. Ambler [Amb98] suggests three types:

a. Stage pattern—*defines a problem (solution) associated with a framework activity for the process.*

Eg: **Establishing Communication.**

b. Task pattern—*defines a problem (Solution) associated with a software engineering action*

E.g., **Requirements Gathering is a task pattern).**

c. Phase pattern—*define the sequence of framework activities that occurs within the process,*

Eg: **SpiralModel or Prototyping.**

4. Initial Context: Conditions under which the pattern applies.

Prior to the initiation of the pattern :

a. What organizational or team-related activities have already occurred?

b. What is the Entry state for the process?

c. Software engineering information or project information already exists?

For example, the Planning pattern requires that :

a. Collaborative communication has been established between customers and software engineers.

b. Successful completion of a number of task patterns for the communication pattern has occurred.

c. The project constraints, basic requirements, and the project scope are known.

5. Problem: Any specific problem is to be solved by pattern.

6. Solution: Describes how to implement pattern successfully.

Process Patterns (Conti)

7. Resulting Context: Describe the conditions that will result once the pattern has been successfully implemented. Upon completion of the pattern:

- What Organizational or team-related activities must have occurred?
- What should be the exit state for the process?
- What software engineering information has been developed?

8. Related pattern: Provide a list of all process patterns that are directly related to the current pattern.

For example, the stage pattern **Communication** encompasses the task patterns: **Project Team, Collaborative Guidelines, Scope Isolation, Requirements Gathering, Constraint Description, and Scenario Creation.**

9. Known uses and Examples: Indicates the specific instances. In which the pattern is applicable.

For example, communication is mandatory at the beginning of every software project, is recommended throughout the software project, and is mandatory once the deployment activity is underway.

Process Patterns (Example)

Pattern Name: RequirementsUnclear

Intent: Requirements are not clear. So aim is to make an model(a prototype)iteratively to solidify the exact requirements.

Type: Phase Pattern

Initial Context: Before going to the prototyping these basic conditions should be made

1. Stakeholder has some idea about their requirements i.e. what they exactly want
2. Communication medium should be established between stakeholder and software development team to ensure proper understanding about the requirements and future product
3. Initial understanding about other factors of project like scope of project, duration of project, budget of project etc.

Problem: hazy and nonexistent requirements.

Solution: A description of the prototyping process should be presented.

Resulting Context: A prototype model which can give a clear idea about the actual product and that needs to be agreed by stakeholder.

Related Patterns: Requirement extraction, Iterative design, customer communication, Iterative development, Customer assessment etc.

Known Uses & Examples: When stakeholder requirements are unclear and uncertain, prototyping is recommended.

PROCESS ASSESSMENT AND IMPROVEMENT

A number of different approaches to software process assessment and improvement have been proposed over the past few decades:

1. **Standard CMMI Assessment Method for Process Improvement (SCAMPI):** provides a five-step process assessment model that incorporates five phases: initiating, diagnosing, establishing, acting, and learning.
2. **CMM-Based Appraisal for Internal Process Improvement (CBA IPI):** provides a diagnostic technique for assessing the relative maturity of a software organization.
3. **SPICE (ISO/IEC15504)**—a standard that defines a set of requirements for software process assessment. The intent of the standard is to assist organizations in developing an objective evaluation of the efficacy of any defined software process [ISO08].
4. **ISO 9001:2000 for Software**—a generic standard that applies to any organization that wants to improve the overall quality of the products, systems, or services that it provides. Therefore, the standard is directly applicable to software organizations and companies.

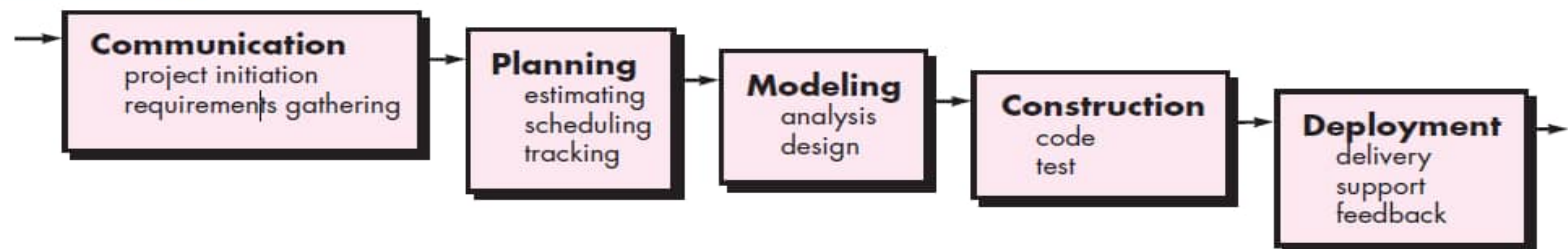
2.3 Prescriptive Process models

- Prescriptive process models were originally proposed to bring order to the chaos of software development.
- These traditional models have brought certain amount of useful structure to software engineering work and have provided a reasonably effective road map for software teams.
- If prescriptive process models strive for structure and order, are they in appropriate for a software world that thrives on change?
- Yet, if we reject traditional process models and replace them with something less structured, do we make it impossible to achieve coordination and coherence in software work?
- We call them “prescriptive” because they prescribe a set of process elements—framework activities, software engineering actions, tasks, work products, quality assurance, and change control mechanisms for each project.

2.3.1 Waterfall model

- It is used when the requirements for a problem are well understood—when work flows from **communication through deployment in a reasonably linear fashion**.
- It is only used when **requirements are well defined and reasonably stable**.
- The *waterfall model*, sometimes called the *classic life cycle*, suggests a **systematic sequential approach to software development** that begins with customer specification of requirements and progresses through planning, modeling, construction, and deployment.

FIGURE 2.3 The waterfall model



Waterfall model(Continued)

the problems that are sometimes encountered when the waterfall model is applied are:

1. **Real projects rarely follow the sequential flow that the model proposes.** Although the linear model can accommodate iteration, it does so indirectly. As a result, changes can cause confusion as the project team proceeds
2. **It is often difficult for the customer to state all requirements explicitly.** The waterfall model requires this and has difficulty accommodating the natural uncertainty that exists at the beginning of many projects.
3. **The customer must have patience.** A working version of the program(s) will not be available until late in the project time span. A major blunder, if undetected until the working program is reviewed, can be disastrous.

Waterfall model(Continued)

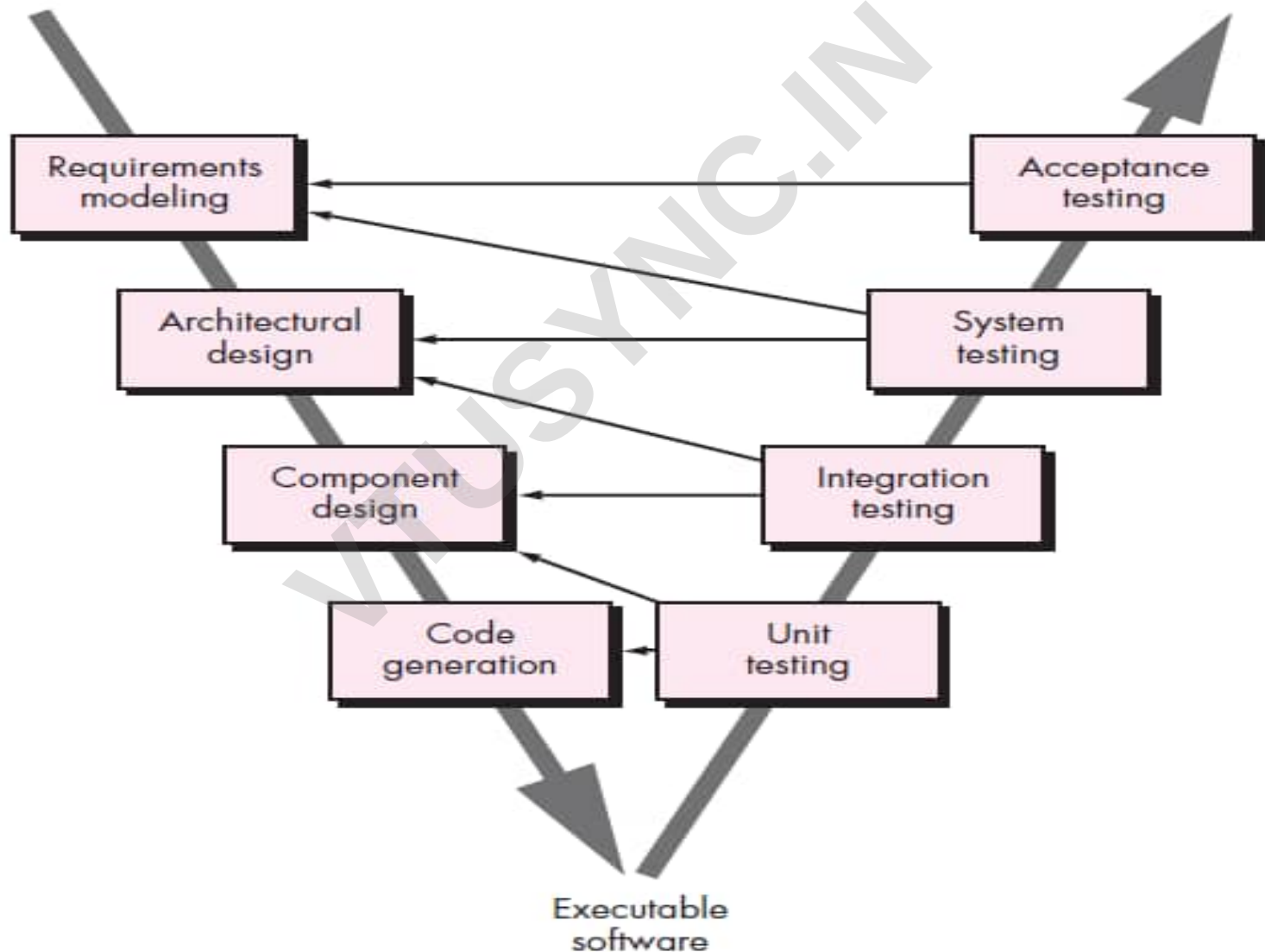
Advantages of waterfall model

- The waterfall model is simple and easy to understand, implement, and use.
- All the requirements are known at the beginning of the project, hence it is easy to manage.
- It avoids overlapping of phases because each phase is completed at once.
- This model works for small projects because the requirements are understood very well.
- This model is preferred for those projects where the quality is more important as compared to the cost of the project.

Disadvantages of the waterfall model

- This model is not good for complex and object oriented projects.
- It is a poor model for long projects.
- The problems with this model are uncovered, until the software testing.
- The amount of risk is high.

The V-model(Verification and validation model)



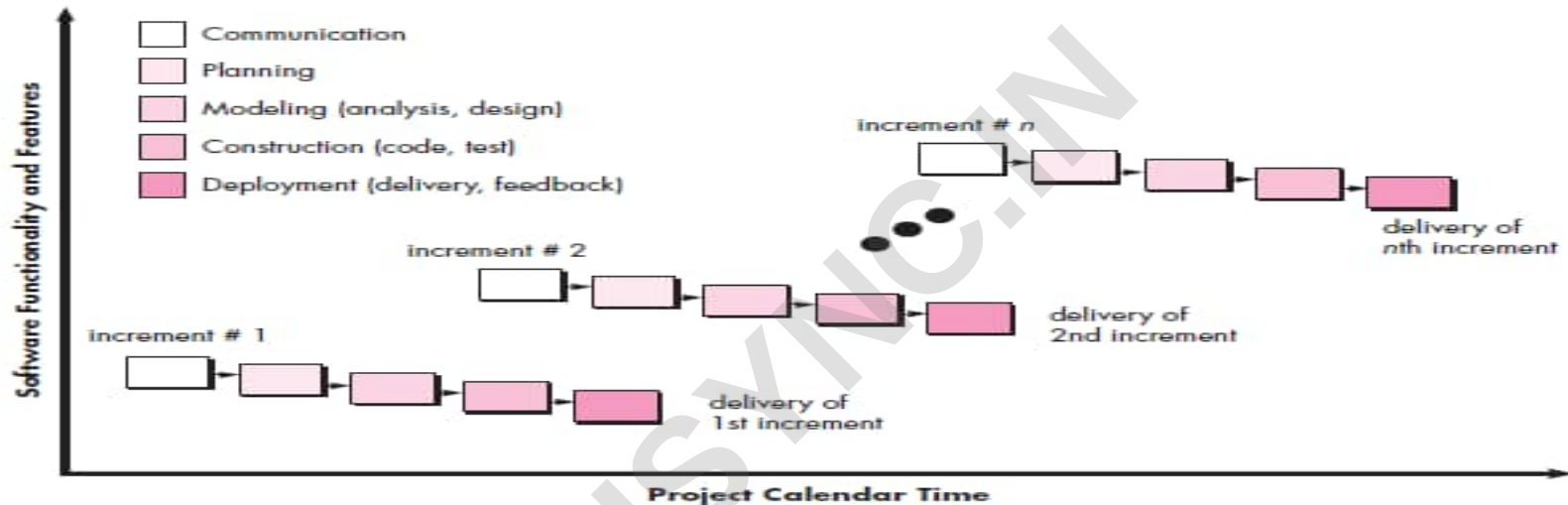
The V- model(Verification and validation model)

- A variation in the representation of the waterfall model is called the *V-model*.
- the V-model depicts the relationship of quality assurance actions to the actions associated with communication, modeling, and early construction activities.
- As a software team moves down the left side of the V, basic problem requirements are refined into progressively more detailed and technical representations of the problem and its solution.
- Once code has been generated, the team moves up the right side of the V, essentially performing a series of tests (quality assurance actions) that validate each of the models created as the team moved down the left side

2.3.2 Incremental Process Models

- There are many situations in which initial software requirements are reasonably well defined, but the overall scope of the development effort precludes a purely linear process.
- There may be compelling need to provide a limited set of software functionality to users quickly and then refine and expand on that functionality in later software releases. In such cases, you can choose a process model that is designed to produce the software in increments.
- The *incremental model combines elements of linear and parallel process flows*
- For example, word-processing software(I increment..... Iv increment)
- When an incremental model is used, the first increment is often a *core product*.
- That is, basic requirements are addressed but many supplementary features (some known, others unknown) remain undelivered.
- The core product is used by the customer(or undergoes detailed evaluation).

Incremental Process Models(Conti)



- **Incremental development is particularly useful when staffing is unavailable** for a complete implementation by the business deadline that has been established for the project. Solution....
- **In addition, increments can be planned to manage technical risks.** For example, a major system might require the availability of new hardware that is under development and whose delivery date is uncertain. Solution:

Incremental Process Models(Conti..)

Advantages of incremental model

- This model is flexible because the cost of development is low and initial product delivery is faster.
- It is easier to test and debug during the smaller iteration.
- The working software generates quickly and early during the software life cycle.
- The customers can respond to its functionalities after every increment.

Disadvantages of the incremental model

- The cost of the final product may cross the cost estimated initially.
- This model requires a very clear and complete planning.
- The planning of design is required before the whole system is broken into small increments.
- The demands of customer for the additional functionalities after every increment causes problem during the system architecture.

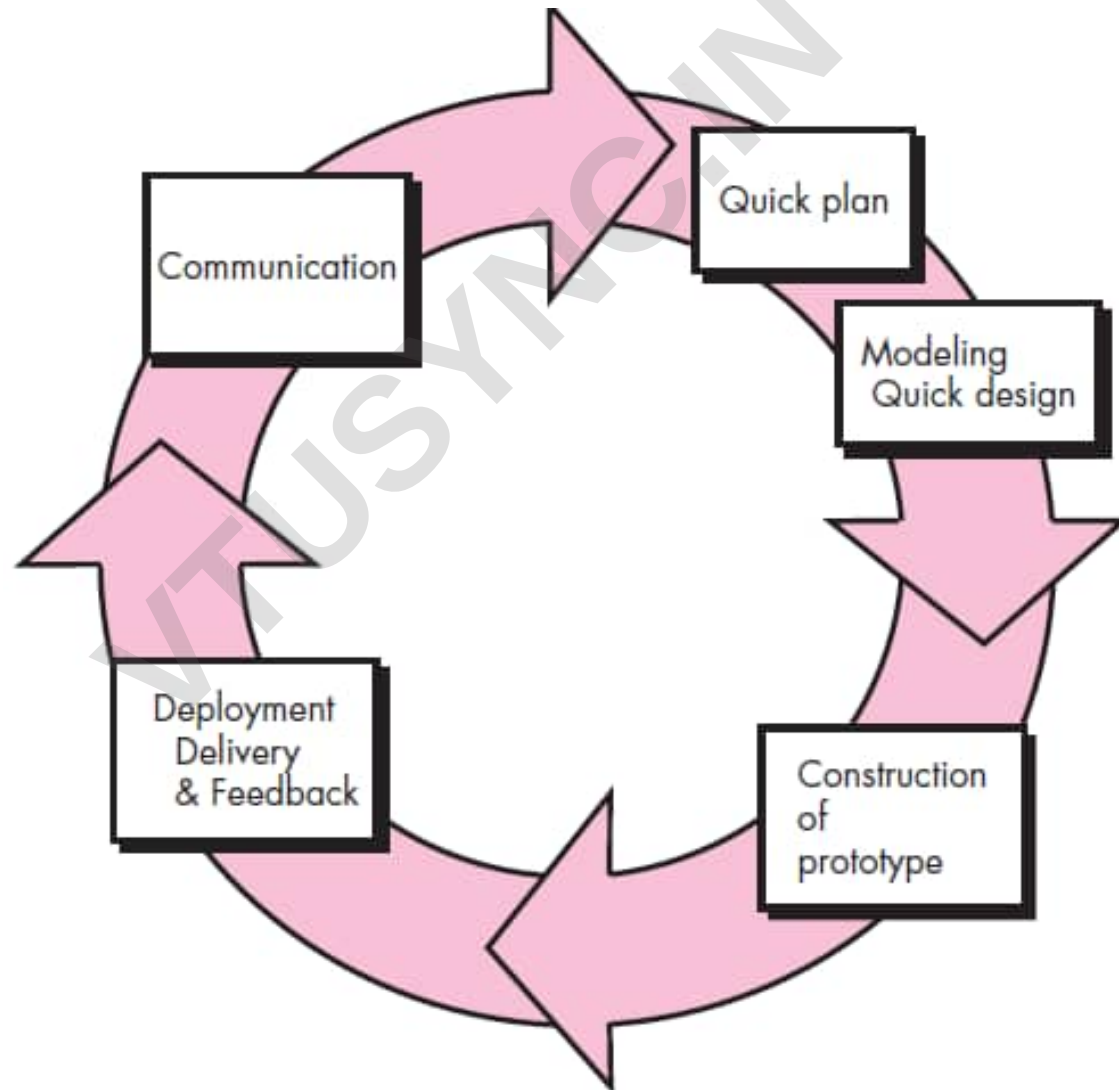
2.3.3 Evolutionary Process Models

- Software, like all complex systems, evolves over a period of time.

Factors

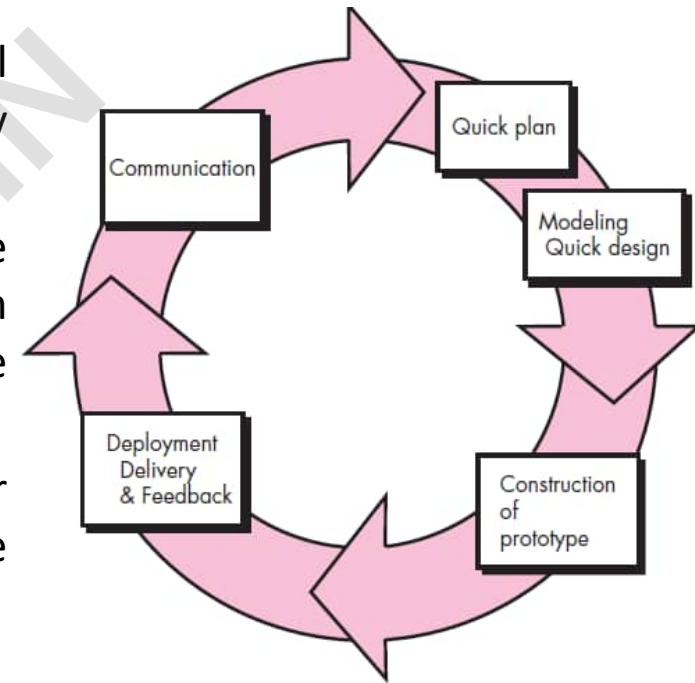
1. **Business and product requirements often change as development proceeds, making a straight line path to an end product unrealistic.**
 2. **tight market deadlines; a limited version must be introduced to meet competitive or business pressure.**
 3. **a set of core product or system requirements is well understood.**
- you need a process model that has been explicitly designed to accommodate a product that evolves over time.
 - Evolutionary models are iterative.
 - They are characterized in a manner that enables you to develop increasingly more complete versions of the software.

Evolutionary Process Models (Prototyping)



Evolutionary Process Models (Prototyping)

- Often, a customer defines a set of general objectives for software, but does not identify detailed requirements for functions and features.
- In other cases, the developer may be unsure of the efficiency of an algorithm, the adaptability of an operating system, or the form that human-machine interaction should take.
- the prototyping paradigm assists you and other stakeholders to better understand what is to be built when requirements are fuzzy.
- The fig
- Ideally, the prototype serves as a mechanism for identifying software requirements.
- The prototype can serve as “the first system.”
- Although some prototypes are built as “**throwaways**,” others are evolutionary in the sense that the prototype slowly evolves into the actual system.



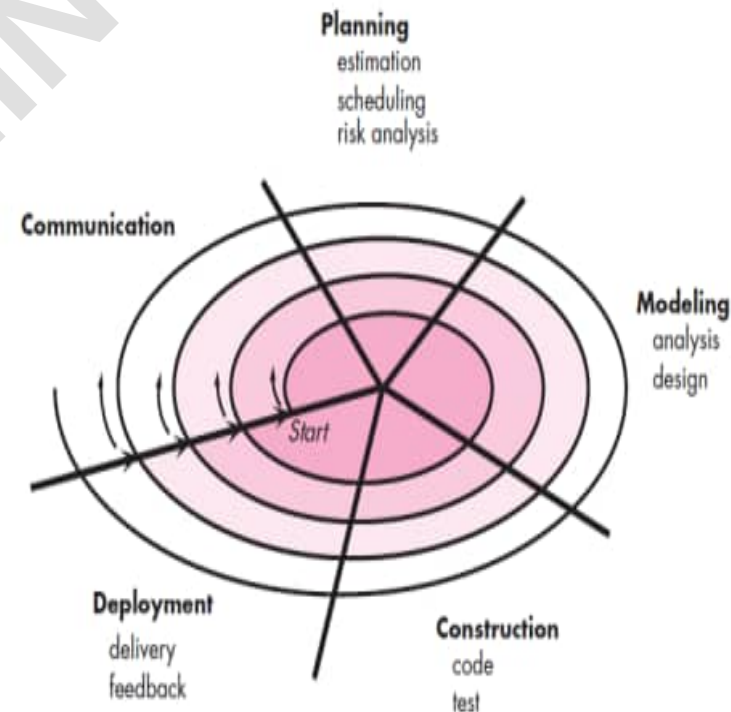
**The
prototyping
paradigm**

Evolutionary Process Models (Prototyping)

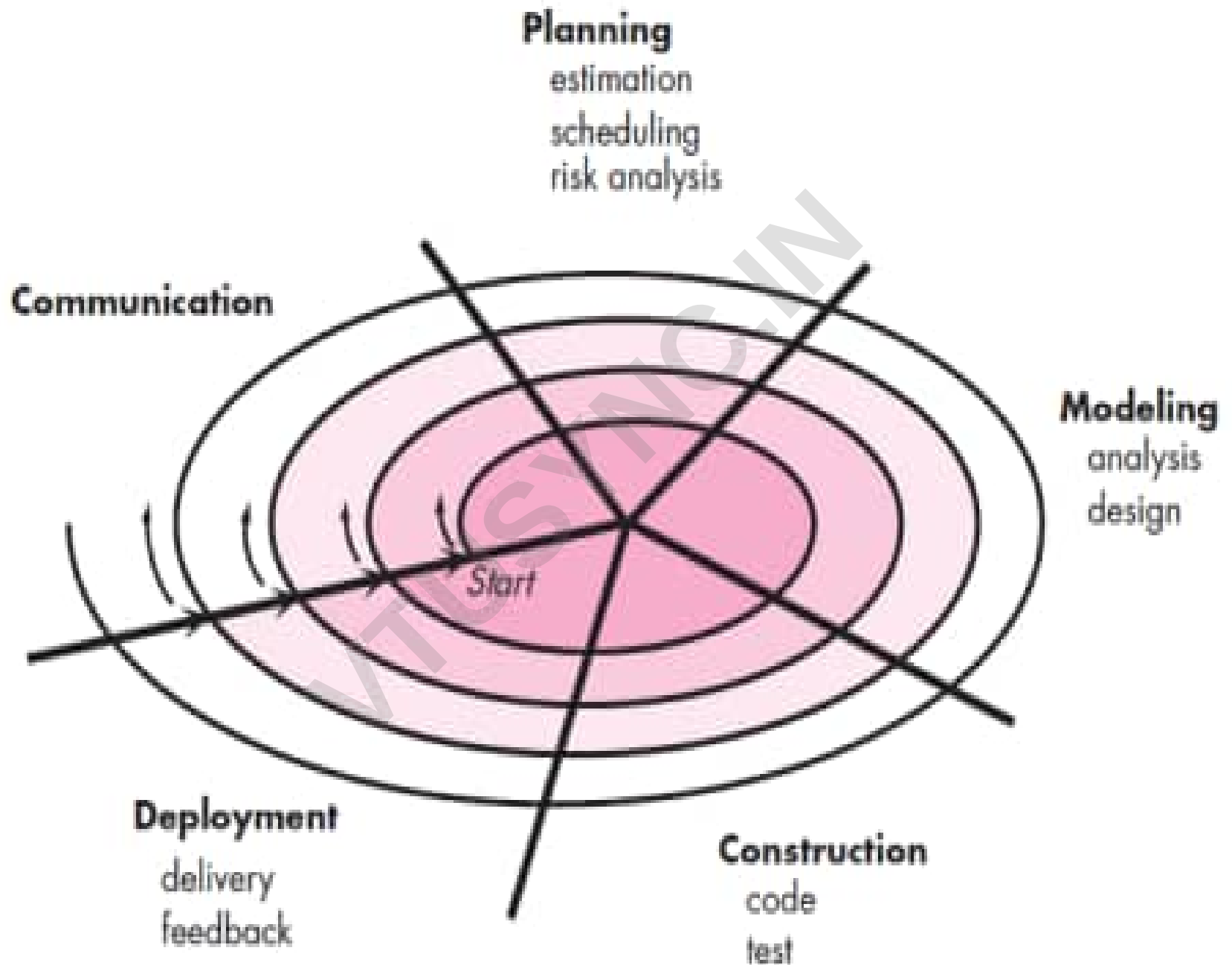
- prototyping can be problematic for the following reasons:
 1. Stakeholders see what appears to be a working version of the software.
 2. As a software engineer, you often make implementation compromises in order to get a prototype working quickly.
- The key is to define the rules of the game at the beginning;
- that is, all stakeholders should agree that the prototype is built to serve as a mechanism for defining requirements.
- It is then discarded (at least in part), and the actual software is engineered with an eye toward quality

Evolutionary Process Models(The Spiral model)

- couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model.
- It provides the potential for rapid development of increasingly more complete versions of the software.
- The fig
- Each pass through the planning region results in adjustments to the project plan.
- Cost and schedule are adjusted based on feedback derived from the customer after delivery.
- In addition, the project manager adjusts the planned number of iterations required to complete the software.



A typical spiral model



Evolutionary Process Models(The Spiral model)

ADVANTAGES

- The spiral model is a realistic approach to the development of large-scale systems and software,
- The spiral model uses prototyping as a risk reduction mechanism
- Enables you to apply the prototyping approach at any stage in the evolution of the product.
- The spiral model demands a direct consideration of technical risks at all stages of the project and, if properly applied, should reduce risks before they become problematic.

DISADVANTAGES

- the evolutionary approach is controllable.
- It demands considerable risk assessment expertise
- If a major risk is not uncovered and managed, problems will undoubtedly occur.

2.3.4 Concurrent Models

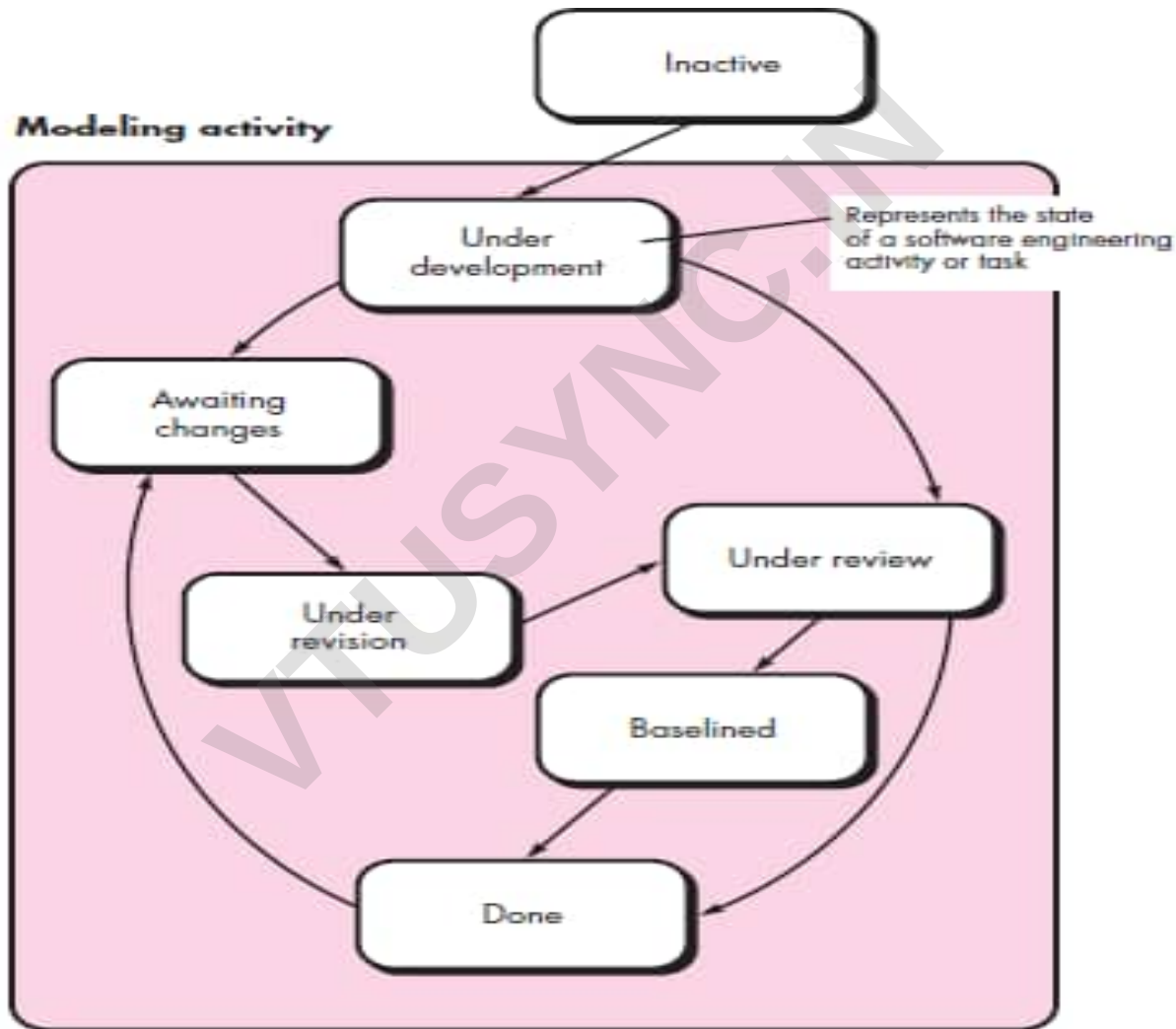


Fig: One element of the concurrent process model

2.3.4 Concurrent Models

- The *concurrent development model*, sometimes called *concurrent engineering*, allows a software team to **represent iterative and concurrent elements of any of the process models**.
- For example, the modeling activity defined for the spiral model is accomplished by invoking one or more of the following software engineering actions: prototyping, analysis, and design
- Working of the concurrent model:...

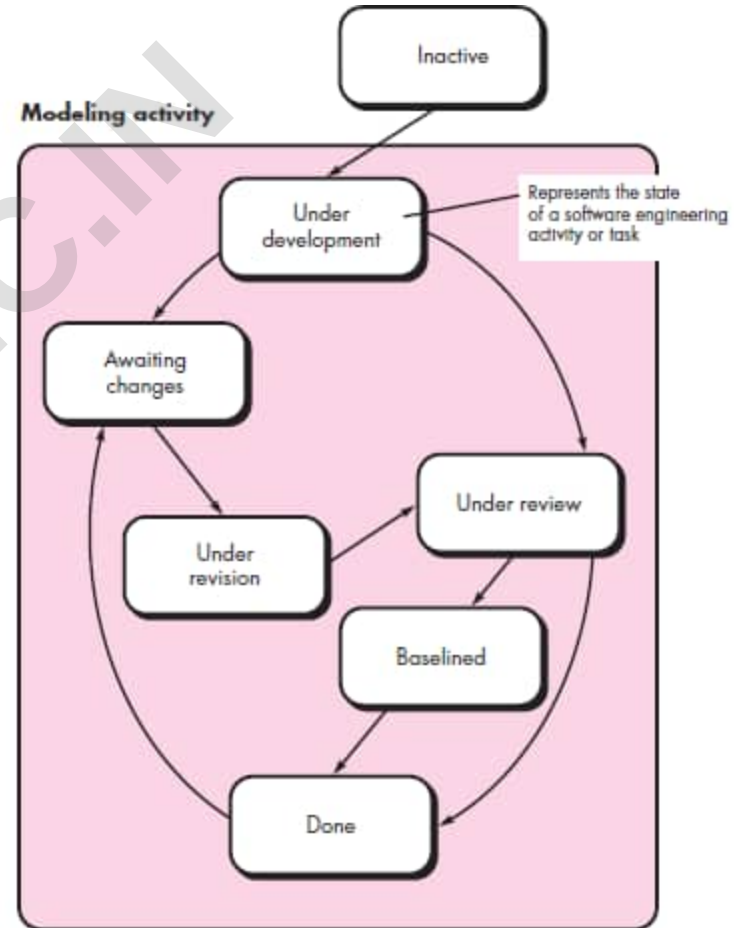


Fig: One element of the concurrent process model

2.4 SPECIALIZED PROCESS MODELS

these models tend to be applied when a specialized or narrowly defined software engineering approach is chosen.

I. Component-Based Development

- Commercial off-the-shelf (COTS) software components, developed by vendors who offer them as products, provide targeted functionality with well-defined interfaces that enable the component to be integrated into the software that is to be built.
- The *component-based development model incorporates many of the characteristics of the spiral model.*
- *It is evolutionary in nature, demanding an iterative approach to the creation of software. However, the component-based development model constructs applications from prepackaged software components.*
- Modeling and construction activities begin with the identification of candidate components.

SPECIALIZED PROCESS MODELS(Conti..)

model incorporates the following steps:

1. Available component-based products are researched and evaluated for the application domain in question.
 2. Component integration issues are considered.
 3. A software architecture is designed to accommodate the components.
 4. Components are integrated into the architecture.
 5. Comprehensive testing is conducted to ensure proper functionality.
- leads to software reuse, and reusability provides software engineers with a number of measurable benefits.

SPECIALIZED PROCESS MODELS(Conti..)

II. The Formal Methods Model

- encompasses a set of activities that leads to formal mathematical specification of computer software.
- Formal methods enable you to specify, develop, and verify a computer-based system **by applying a rigorous, mathematical notation**.
- A variation on this approach, called ***clean room software engineering*** is currently applied by some software development organizations.
- When formal methods are used during development, they provide a mechanism for **eliminating many of the problems** that are difficult to overcome using other software engineering paradigms.
- When formal methods are used during design, they **serve as a basis for program verification** and therefore enable you to discover and correct errors that might otherwise go undetected.
- concern about its applicability in a business environment has been voiced:
 1. The development of formal models is currently **quite time consuming and expensive**.
 2. Because few software developers have the necessary background to apply formal methods, **extensive training is required**.
 3. It is difficult to use the models as a communication mechanism for technically unsophisticated customers.

SPECIALIZED PROCESS MODELS(Conti..)

III. Aspect-Oriented Software Development

- As modern computer-based systems become more sophisticated (and complex), certain *concerns*—*customer required properties or areas of technical interest*—span the entire architecture.
- Some concerns are high-level properties of a system (e.g., security, fault tolerance). Other concerns affect functions (e.g., the application of business rules), while others are systemic (e.g., task synchronization or memory management).
- When concerns cut across multiple system functions, features, and information, they are often referred to as *crosscutting concerns*.
- AOSD/AOP provides a process and methodological approach for defining, specifying, designing, and constructing *aspects*—“*mechanisms beyond subroutines and inheritance for localizing the expression of a crosscutting concern*”.
- A distinct aspect-oriented process has not yet matured. However, it is likely that such a process will adopt characteristics of both evolutionary and concurrent process models.
- The evolutionary model is appropriate as aspects are identified and then constructed.
- The parallel nature of concurrent development is essential because aspects are engineered independently of localized software components and yet, aspects have a direct impact on these components.

2.5 The Unified Process

- In their seminal book on the *Unified Process*, Ivar Jacobson, Grady Booch, and James Rumbaugh [Jac99] discuss the need for a “use case driven, architecture-centric, iterative and incremental”
- Unified Process is an attempt to draw on the best features and characteristics of traditional software process models, but implements many of the best principles of agile software development
- The Unified Process recognizes the importance of **customer communication and streamlined** methods for describing the customer’s view of a system (the use case).
- It emphasizes the **important role of software architecture** and helps the architect focus on the right goals, such as understandability, reliance to future changes, and reuse
- It suggests a process flow that is **iterative and incremental, providing the evolutionary feel** that is essential in modern software development.

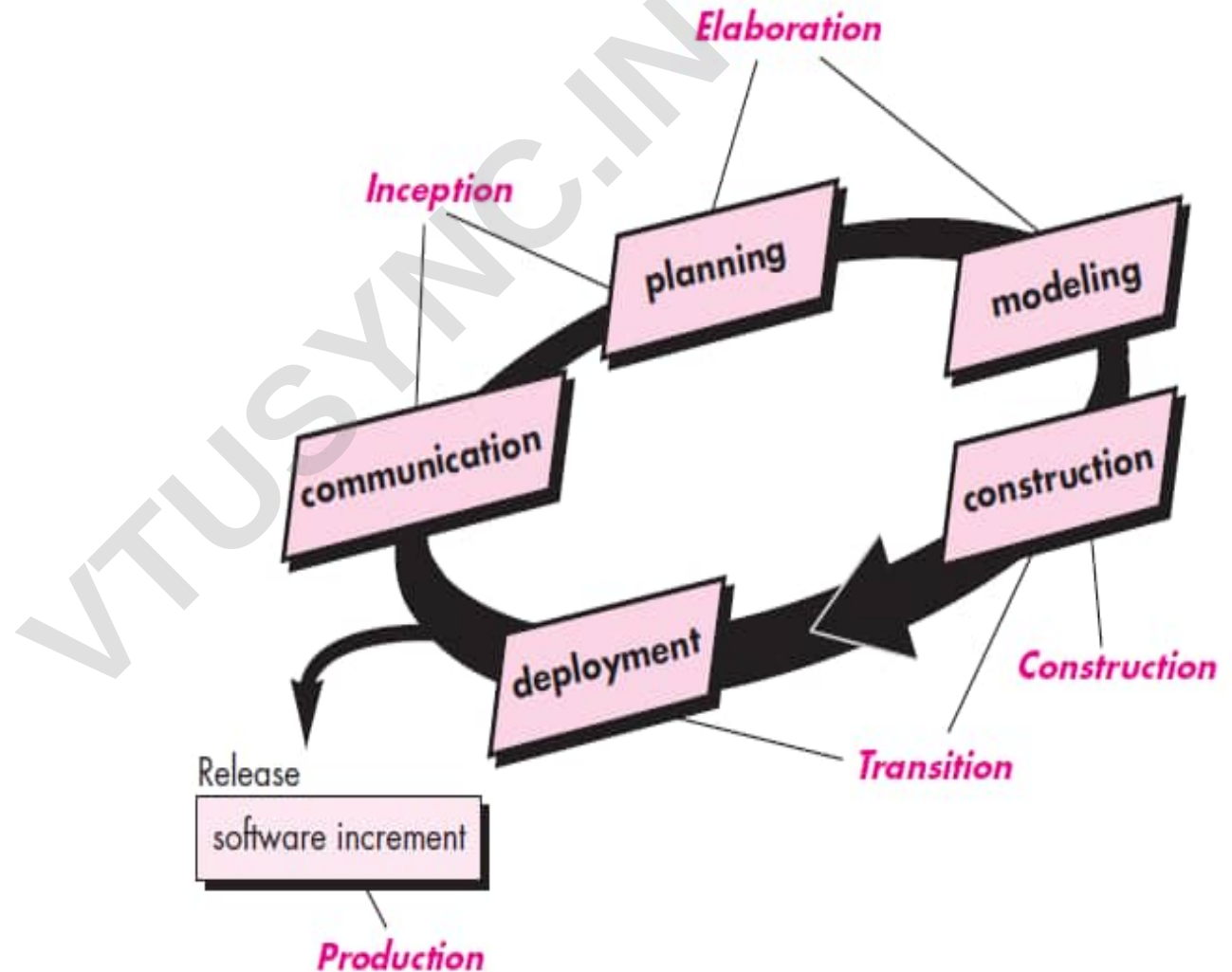
2.5.1 A Brief History

- During the early 1990s James Rumbaugh , Grady Booch , and Ivar Jacobson began working on a “unified method” that would combine the best features of each of their individual object-oriented analysis and design methods and adopt additional features proposed by other expert in object-oriented modeling.
- The result was UML—a *unified modeling language that contains a robust* notation for the modeling and development of object-oriented systems.
- By 1997, UML became a de facto industry standard for object-oriented software development.
- Over the next few years, Jacobson, Rumbaugh, and Booch developed the *Unified Process, a framework for object-oriented software* engineering using UML.

2.5.2 Phases of the Unified Process

FIGURE 2.9

The Unified
Process



2.5.2 Phases of the Unified Process (Conti..)

1. **The inception phase** of the UP encompasses both customer communication and planning activities.
 - By collaborating with stakeholders, business requirements for the software are identified; a rough architecture for the system is proposed; and a plan for the iterative, incremental nature of the ensuing project is developed.
 - **Fundamental business requirements** are described through a set of preliminary use cases that describe which features and functions each major class of users desires.
 - **Architecture** at this point is nothing more than a tentative outline of major subsystems and the function and features that populate them.
 - Later, the architecture will be refined and **expanded into a set of models** that will represent different views of the system.
 - **Planning identifies** resources, assesses major risks, defines a schedule, and establishes a basis for the phases that are to be applied as the software increment is developed.

2.5.2 Phases of the Unified Process (Conti..)

2. The *elaboration phase* encompasses the communication and modeling activities of the generic process model

- Elaboration refines and expands the preliminary use cases that were developed as part of the inception phase and expands the architectural representation to include five different views of the software—the use case model, the requirements model, the design model, the implementation model, and the deployment model.
- In some cases, elaboration creates an “executable architectural baseline” [Arl02] that represents a “first cut” executable system. The
- The plan is carefully reviewed at the culmination of the elaboration phase to ensure that scope, risks, and delivery dates remain reasonable.

2.5.2 Phases of the Unified Process (Conti..)

3. The *construction phase* of the UP is identical to the construction activity defined for the generic software process.

- Using the architectural model as input, the construction phase develops or acquires the software components that will make each use case operational for end users.
- To accomplish this, requirements and design models that were started during the elaboration phase are completed to reflect the final version of the software increment.
- All necessary and required features and functions for the software increment (i.e., the release) are then implemented in source code.
- As components are being implemented, unit tests are designed and executed for each.
- Integration activities (component assembly and integration testing) are conducted.
- Use cases are used to derive a suite of acceptance tests that are executed prior to the initiation of the next UP phase.

2.5.2 Phases of the Unified Process (Conti..)

4. The *transition phase*: *of the UP encompasses the latter stages of the generic construction activity and the first part of the generic deployment (delivery and feedback) activity.*

- Software is given to end users for beta testing and user feedback reports both defects and necessary changes.
- The software team creates the necessary support information (e.g., user manuals, troubleshooting guides, installation procedures) that is required for the release.

5. The *production phase* : *of the UP coincides with the deployment activity of the generic process.*

- The ongoing use of the software is monitored, support for the operating environment (infrastructure) is provided, and defect reports and requests for changes are submitted and evaluated.

2.5.2 Phases of the Unified Process (Conti..)

- Five UP phases do not occur in a sequence, but rather with staggered concurrency.
- A software engineering workflow is distributed across all UP phases.
- It should be noted that not every task identified for a UP workflow is conducted for every software project. The team adapts the process (actions, tasks, subtasks, and work products) to meet its needs.

Personal and Team Process Models

- The best software process is one that is close to the people who will be doing the work
 - In an ideal setting, you would create a process that best fits your needs, and at the same time, meets the broader needs of the team and the organization
 - Watts Humphrey ([Hum97] and [Hum00]) argues that it is possible to create a “personal software process” and/or a “team software process.” Both require hard work, training, and coordination, but both are achievable
- 1. Personal Software Process (PSP):** emphasizes personal measurement of both the work product that is produced and the resultant quality of the work product. The PSP model defines five framework activities:
- a. Planning b. High level design c. High level design review
 - d. Development e. Postmortem

Personal and Team Process Models(Conti..)

PSP (Conti..)

- a. **Planning:** This activity isolates requirements and develops both size and resource estimates, a defect estimate is made. Finally, development tasks are identified and a project schedule is created.
- b. **High-level design:** External specifications for each component to be constructed are developed and a component design is created. Prototypes are built when uncertainty exists. All issues are recorded and tracked.
- c. **High-level design review:** Formal verification methods are applied to uncover errors in the design. Metrics are maintained for all important tasks and work results.
- d. **Development:** The component-level design is refined and reviewed. Code is generated, reviewed, compiled, and tested. Metrics are maintained ...
- e. **Postmortem:** Using the measures and metrics collected (this is a substantial amount of data that should be analyzed statistically), the effectiveness of the process is determined. Measures and metrics should provide guidance for modifying the process to improve its effectiveness.

Personal and Team Process Models(Conti..)

(PSP Continued..)

- PSP stresses the need to identify errors early and, just as important, to understand the types of errors that you are likely to make.
- PSP represents a disciplined, metrics-based approach to software engineering that may lead to culture shock for many practitioners.
- when PSP is properly introduced to software engineers [Hum96], the resulting improvement in software engineering productivity and software quality are significant.
- PSP is intellectually challenging and demands a level of commitment (by practitioners and their managers) that is not always possible to obtain.
- Training is relatively lengthy, and training costs are high.

Personal and Team Process Models

2. Team Software Process (TSP): The goal of TSP is to build a “self directed” project team that organizes itself to produce high-quality software. Humphrey [Hum98] defines the following objectives for TSP:

- a) Build self-directed teams that plan and track their work, establish goals, and own their processes and plans. These can be pure software teams or integrated product teams (IPTs) of 3 to about 20 engineers.
- b) Show managers how to coach and motivate their teams and how to help them sustain peak performance.
- c) Accelerate software process improvement by making CMM Level 5 behavior normal and expected.
- d) Provide improvement guidance to high-maturity organizations.
- e) Facilitate university teaching of industrial-grade team skills
- TSP defines the following framework activities: **project launch, high-level design, implementation, integration and test, and postmortem.**

Team Software Process (TSP) (Conti)

- these activities enable the team to plan, design, and construct software in a disciplined manner while at the same time quantitatively measuring the process and the product. The postmortem sets the stage for process improvements.
- TSP makes use of a wide variety of scripts, forms, and standards that serve to guide team members in their work.
- TSP recognizes that the best software teams are self-directed.
- Like PSP, TSP is a rigorous approach to software engineering that provides distinct and quantifiable benefits in productivity and quality.
- The team must make a full commitment to the process and must undergo thorough training to ensure that the approach is properly applied.